

Actividad 2 de NodeJS
Backend de autenticación — APREHENDER

Lorly Fuentes
Desarrollo Web — Bit
Docente: Sebastián Amaya
Octubre de 2026

Enlaces de entrega

Video de pruebas en Postman: [Reunión con LORLY YANINI FUENTES ROMERO-20261005_010921-Grabación de la reunión.mp4](#)

Repositorio en GitHub: <https://github.com/lorlyfuentes/aprehender-backend>

Resumen ejecutivo

1. Descripción de la aplicación

APREHENDER es una herramienta pensada en aquellos estudiantes de transición a grado quinto de primaria que tienen problemas de dislexia y disgrafía. Va dirigida a padres de familia, docentes e instituciones.

2. Objetivo general del proyecto

Fortalecer el proceso lecto-escritor en los estudiantes de transición a quinto con dislexia y disgrafía a través de una herramienta gamificada con ejercicios interactivos que promuevan en el estudiante un aprendizaje autónomo.

3. Objetivos del backend

- Permitir el registro e ingreso de los usuarios y revisar el estado del pago del usuario para dar o negar acceso.
- Proteger y garantizar la confidencialidad de la información, en donde la contraseña se guarda encriptada, nunca se devuelve en las respuestas, y el acceso al perfil requiere un token.
- Garantizar que los datos que llegan en el registro y el login sean válidos antes de procesarlos.

4. Funcionalidades

Registro

Los datos que recibe son el nombre (nombre completo del usuario), el correo, la contraseña y el rol. Con estos datos valida que se haya ingresado un nombre, es decir, que no quede vacío o sin diligenciar; que se ingrese un correo válido; que la contraseña tenga mínimo 8 caracteres, y que el rol sea docente, familia o institución. Si toda la información ingresada es correcta, se encripta la contraseña con bcrypt y se guarda el usuario en MongoDB con el estado de pago "pendiente". El resultado en Postman será 201, "Usuario creado correctamente". Por el contrario, si los datos están errados, en Postman responde 400 con la lista de errores: "El nombre es obligatorio", "El correo no es válido", "La contraseña debe tener mínimo 8 caracteres" o "El rol no es válido". Si el correo ya está registrado, responderá 409, "El correo ya está registrado", y 500, "Error al crear el usuario", cuando hay una falla en el servidor.

Login

En el login, los datos que recibe son el correo y la contraseña. Si falta algún dato o el correo no tiene un formato válido, responde 400: "El correo no es válido" o "La contraseña es obligatoria". Cuando los datos son válidos, se busca el usuario en MongoDB por el correo y se compara la contraseña escrita con la encriptada usando bcrypt. Si esta coincide, genera el token con jsonwebtoken, que lleva el id y el rol del usuario y tiene caducidad de 1 hora. Habiendo hecho esto, responde 200 con el mensaje "Inicio de sesión exitoso" y el token. Responde 401 cuando el correo o la contraseña son incorrectos, con el mismo mensaje en ambos casos para no revelar qué correos están registrados, y 500, "Error al iniciar sesión", cuando hay una falla en el servidor.

Perfil (ruta protegida)

Inicialmente, el token se toma y se pega en la Authorization con el formato Bearer. Aquí el middleware valida si se envió el token; si no se envió, o si es falso o venció, el resultado es 401. Si es válido, saca el id y deja pasar a la ruta. Si después de validar en MongoDB se evidencia que el usuario no existe, responde 404, "Usuario no encontrado". Si el pago del usuario no está activo, genera el mensaje 403, "Acceso denegado: el pago no está activo". Por el contrario, si está activo, responde 200 con los datos del usuario, sin la contraseña, y si hay falla en el servidor, 500, "Error al obtener el perfil".

5. Tecnologías utilizadas

- **Node.js y Express:** Node.js ejecuta el código JavaScript en el servidor, y Express crea el servidor y las rutas (/registro, /login, /perfil).
- **MongoDB Atlas y Mongoose:** Atlas es la base de datos en la nube donde se guardan los usuarios, y Mongoose es la librería para definir el modelo de usuario, guardar y buscar datos.
- **bcryptjs:** Encripta la contraseña al registrar y se usa en el login para comparar la contraseña escrita con la encriptada.
- **express-validator:** Revisa que los datos sean válidos y, si no, responde con el código 400 con los errores.
- **jsonwebtoken:** Genera y verifica el token en el middleware.
- **Postman:** Herramienta que se usa para probar los endpoints, enviando peticiones y viendo las respuestas del servidor.
- **Git y GitHub:** Git es el sistema de control de versiones: guarda el historial del proyecto con ramas y commits. GitHub es el repositorio en la nube. Aquí se realizan los Pull Requests para fusionar las ramas. Queda consolidado todo el proyecto.

6. Estado actual y próximos pasos

El backend en este momento se encuentra funcional frente a la parte de registro, login y perfil. Se realizaron diferentes validaciones en Postman ingresando datos errados, como nombres vacíos, contraseñas menores a 8 caracteres, correos ya registrados o con formatos errados, y la ruta protegida sin token (401), con pago pendiente (403) y activo (200). Falta implementar la parte del frontend. El próximo paso sería mostrar contenido o actividades distintas según el rol (docente, familia o institución). El estado de pago actualmente se realiza manualmente en Atlas; el próximo paso es crear la forma de que esta actualización se realice desde la aplicación.